

Paradigmas de solução de problemas

Seja em programação competitiva ou na área de algoritmos em geral, existem os paradigmas de solução de problemas, que são técnicas que guiam a forma de pensar na solução do problema.

Nesse tópico, alguns dos mais comuns paradigmas:

- **Busca completa (força bruta):** Consiste em, dado um problema, testar todas soluções possíveis até se encontrar a correta. Essa garante sempre encontrar a resposta correta mas geralmente tem complexidade muito alta, funcionando apenas para problemas pequenos.
- **Algoritmos gulosos:** Constrói apenas uma solução de pouco em pouco, escolhendo sempre o elemento que parece ser o melhor naquele momento sem se preocupar com o futuro. Não funciona para todos os problemas, mas geralmente é eficiente quando funciona.
- **Divisão e conquista:** Estratégia que resolve o problema com 3 passos: dividir, conquistar e combinar. Primeiro o problema total é dividido em subproblemas menores. Cada subproblema é resolvido de forma independente.
- **Programação dinâmica:** Semelhante à divisão e conquista, divide o problema em subproblemas menores onde esses subproblemas podem se repetir (sobreposição). Utiliza de mecanismos de memorização dos resultados para evitar cálculos repetidos.

Exemplos de problemas

Esses são paradigmas comuns que muitas vezes são combinados para formar a base para algoritmos avançados em diversas áreas da maratona. São alguns exemplos:

- **Algoritmos de Dijkstra:** Pode ser usado para calcular a menor rota de um endereço até outro outro em uma cidade, utiliza o paradigma de programação dinâmica e guloso.
- **Ordenação rápida (Quicksort ou Mergesort):** Utiliza o paradigma de divisão e conquista para ordenar elementos em $O(n \cdot \log(n))$.
- **Problema da mochila (Knapsack):** Utiliza o paradigma de programação dinâmica para encontrar a melhor combinação de itens que cabem em uma mochila com capacidade limitada.
- **Problema do caixeiro viajante (TSP):** Utiliza o paradigma de busca completa para encontrar a rota mais curta que passa por todas as cidades uma vez e retorna à cidade de origem. Pode ser otimizado com programação dinâmica.